

MÔ TẢ THIẾT KẾ SOLVER SOKOBAN

Nhóm 13

Đỗ Hải Đăng – 25020112
Nguyễn Mạnh Thắng – 25023401
Nguyễn Kiên Cường – 25021662
Phạm Yến Nhi – 25023353

Ngày 12 tháng 11 năm 2026

Mục lục

1 Thiết kế **3**
1.1 Cách tiếp cận chung . . . 3
1.2 Tổ chức chương trình . . . 4
1.3 Các thành phần của solver nộp (`solution.py`) . . . 4
1.4 Lý do lựa chọn . . . 5

2 Chất lượng lời giải **5**

3 Thực nghiệm **6**
3.1 Thiết lập . . . 6
3.2 So sánh BFS, UCS, A*, Macro A* . . . 7
3.3 Khảo sát hệ số w cho hai thuật toán A* . . . 10
3.4 Tập trung vào Macro A*: các map khó . . . 12
3.5 Các deadlock mà bộ phát hiện bắt được . . . 14

4 Hạn chế và nguồn tham khảo **17**

Danh sách hình vẽ

1 Cột chồng trạng thái PASS / TIMEOUT / Giới hạn mở rộng theo từng thuật toán . . . 7
2 Phân phối thời gian giải có tính phạt (20 giây cho trường hợp không PASS) . . . 7
3 Bản đồ nhiệt thời gian giải có tính phạt theo từng bản đồ (`test00`–`test19`) . . . 8
4 So sánh độ dài lời giải (tổng số bước robot) trên từng bản đồ PASS . . . 8
5 Quy mô không gian tìm kiếm: số trạng thái mở rộng theo thang log . . . 8
6 Gia tốc thời gian theo từng bản đồ: Macro A* so với A* bước đơn trên các map cùng PASS . . . 9
7 Tương quan giữa thời gian giải và số bước đi (kích thước điểm tỷ lệ với số thùng) 9
8 Khả năng giải đúng (PASS rate) khi tăng trọng số heuristic w . . . 10

9 Phân tích độ nhạy w: trung vị thời gian phạt theo hệ số w 11
10 Đánh đổi chất lượng lời giải (số bước) theo w trên cùng tập map PASS 11
11 Bản đồ nhiệt độ nhạy $w \times$ map cho A* bước đơn và Macro A* 12
12 Xếp hạng bản đồ khó dựa trên số thuật toán không PASS dưới ngưỡng 20 giây . 13
13 Đường giải Macro A* qua 4 mốc (đầu, 1/3, 2/3, kết thúc) trên `test18`, `test09`, `test17` 14
14 Tiến trình số thùng nằm đúng ô đích theo từng bước di chuyển của robot 14
15 Trước và sau một phép đẩy gây deadlock (viền đỏ là thùng vừa bị đẩy, × là ô chết tĩnh) 16
16 Phân bố các ô chết tĩnh trên 6 bản đồ khó nhất 17

Danh sách bảng

1 Tổ chức các tệp mã nguồn trong dự án 4
2 Các dạng deadlock được bộ phát hiện xử lý 5
3 So sánh độ dài lời giải trên một số bản đồ mẫu 6
4 Kết quả thực nghiệm tổng hợp của 4 thuật toán 7
5 Kết quả khảo sát tham số w cho A* và Macro A* 10
6 Danh sách các bản đồ khó nhất trong bộ kiểm thử 12
7 Một số ví dụ deadlock được phát hiện trên các bản đồ khó 15

Tài liệu này giải thích bài nộp `solution.py` (Macro Push Weighted A*) và các thực nghiệm so sánh với BFS, UCS, Weighted A* trên từng bước đi. Mọi số liệu lấy từ `test.ipynb`; các hình nằm trong thư mục `figures/`.

1 Thiết kế

1.1 Cách tiếp cận chung

Chi phí của lời giải là **tổng số bước robot** (đi bộ + đẩy thùng), nên không gian tìm kiếm "mỗi bước một ô" rất lớn: nhiều trạng thái chỉ khác nhau ở chỗ robot đi lòng vòng. Nhóm chia thành hai lớp:

- **Lớp đối chứng (mức bước đơn):** BFS, UCS và Weighted A* chạy trực tiếp trên `state.successors()` của framework. Lớp này giúp kiểm tra chéo luật sinh trạng thái và làm mốc so sánh.
- **Lớp nộp bài (mức macro-push):** Weighted A* tìm kiếm trên các *hành động đẩy gộp*. Mỗi nút của đồ thị là một cấu hình thùng sau một lần đẩy, đoạn robot đi bộ tới vị trí đẩy được tính bằng BFS. Sau khi tìm xong, chuỗi macro được **dựng lại (reconstruct)** thành chuỗi bước cơ bản đúng luật để trả về qua `parent/action/gval`.

1.2 Tổ chức chương trình

File	Vai trò
`solution.py`	**File nộp.** Gồm `MapInfo` (tiền xử lý), phát hiện deadlock, heuristic, `macro_successors`, `reconstruct`, `macro_astar`, `solve`
`BFS.py`, `AStar.py`, `AStarW.py`	Bản đối chứng: BFS, A*/Weighted A* trên bước đơn
`AdvancedHeuristic.py`, `RobotHeuristic.py`, `BasicBoxHeuristic.py`	Các heuristic cho bản A* bước đơn
`mapper.py`	Tiền xử lý bản đồ và `is_deadlock` dùng cho A* bước đơn
`macro_push.py`	Bản macro-push dùng trong giai đoạn thử nghiệm
`benchmark.py`, `tune_w.py`, `test.ipynb`	Chạy thực nghiệm và vẽ biểu đồ

Bảng 1: Tổ chức các tệp mã nguồn trong dự án

1.3 Các thành phần của solver nộp (`solution.py`)

a) Tiền xử lý bản đồ (`MapInfo`).

- *Ô chết tĩnh (static dead squares):* chạy BFS ngược (pull search) từ các ô đích; ô nào không thể kéo ngược tới thì đặt thùng vào đó không bao giờ tới được đích.
- *Ma trận khoảng cách đẩy:* BFS ngược từ mỗi ô đích, cho khoảng cách đẩy thùng thực tế quanh tường (chính xác hơn Manhattan).
- *Hành lang đích, hàng/cột sát biên, tunnel:* dùng để kiểm tra deadlock nâng cao và để gộp các lần đẩy.

b) Phát hiện deadlock (`is_deadlock`, có cache theo tập thùng).

Dạng	Ý nghĩa
Ô chết tĩnh	Thùng nằm trên ô không thể đưa tới đích
Khối 2×2	Bốn ô gồm tường/thùng chưa xong tạo thành khối khóa
Freeze (đóng băng)	Trạng thái bất động lan truyền theo trục X/Y từ tường sang các thùng kề nhau
Bít cửa hành lang đích	Thùng ngoài chặn lối vào đích bên trong
Line/wall	Số thùng trên hàng/cột sát biên vượt số ô đích trên hàng/cột đó

Bảng 2: Các dạng deadlock được bộ phát hiện xử lý

c) Heuristic. $h(n)$ = chi phí ghép cặp 1–1 tối ưu giữa thùng và đích bằng thuật toán Hungarian $O(V^3)$ trên ma trận khoảng cách đẩy. Nếu một thùng không thể tới đích nào thì $h = \infty$ và nhánh bị cắt. Hỗ trợ trường hợp số đích nhiều hơn số thùng.

d) Macro push (`macro_successors`). Với mỗi thùng và mỗi hướng, kiểm tra ô sau thùng và ô đứng đẩy hợp lệ, rồi sinh:

- đẩy một ô;
- *line push*: đẩy liên tiếp theo một hướng tới các ô đích dọc đường;
- *tunnel macro*: khi vào hành lang rộng 1 ô thì đẩy xuyên suốt tới điểm quyết định, trong một macro duy nhất.

Với nhiều robot, hàm xét robot nào tới được vị trí đẩy (flood fill có tính robot khác là vật cản) và có xử lý nhường đường.

e) Tìm kiếm (`macro_astar`). $f(n) = g(n) + w \cdot h(n)$, hàng đợi ưu tiên `heapq`, bảng `explored` lưu g nhỏ nhất theo `hashable_state()`. Mỗi nút sinh ra đều qua `is_deadlock` và kiểm tra $h = \infty$ trước khi vào hàng đợi. Khi chạm trạng thái đích thì gọi `reconstruct`; nếu dựng lại thất bại thì bỏ nhánh đó và đi tiếp. Dừng và trả `False` khi còn khoảng 1,5 giây so với `timebound`.

f) Điểm vào `solve`. Tạo `MapInfo`, dừng sớm (đã ở đích, thùng nhiều hơn đích, không có robot, thùng đang ở ô chết, deadlock ban đầu), rồi chạy `macro_astar`. Mặc định $w = 2.3$; với bản đồ có hơn 4 robot thì dùng $w \geq 2.8$. Không sửa `initial_state`.

1.4 Lý do lựa chọn

- A* có heuristic ghép cặp cho hướng tìm kiếm tốt hơn mù, còn $w > 1$ ưu tiên heuristic để vượt các bản đồ lớn trong giới hạn thời gian.
- Gộp các bước đi và đẩy thành macro giảm mạnh số trạng thái phải mở rộng (xem Mục 3).
- Cắt tỉa deadlock sớm tránh phí thời gian vào nhánh chắc chắn thất bại.
- Giữ BFS/UCS/A* bước đơn làm đối chứng để phát hiện lỗi sinh trạng thái và đo lợi ích thực sự của macro.

2 Chất lượng lời giải

Phương pháp tìm và cải thiện lời giải.

- Đoạn đi bộ trong mỗi macro được dựng bằng BFS nên là đường ngắn nhất cho riêng đoạn đó.
- Tham số w điều khiển đánh đổi giữa tốc độ và độ dài lời giải; nhóm khảo sát $w \in \{1, 2, 3, 4\}$ (Mục 3.3).
- Solver **không** có bước hậu xử lý rút ngắn đường đi sau khi tìm xong.

Về tính tối ưu: nhóm không tuyên bố lời giải nộp là tối ưu.

- BFS và UCS trên bước đơn (chi phí 1 mỗi bước) cho lời giải ngắn nhất *khi chúng giải xong*; đây là mốc tối ưu để so sánh.
- Weighted A* với $w > 1$ ưu tiên heuristic nên không bảo đảm tối ưu.
- Với Macro A*, g là chi phí các macro-transition còn h chỉ ước lượng quãng đẩy thùng (không tính đoạn robot đi bộ), nên không có cận chất lượng chặt chẽ.

Bằng chứng thực nghiệm. Trên các map mà BFS giải được, lời giải Macro A* ($w = 3$) bằng hoặc dài hơn một chút, ví dụ:

Map	BFS/UCS (tối ưu)	A* ($w = 3$)	Macro A* ($w = 3$)
test00	16 bước	18	18
test01	4	5	4
test02	21	21	21
test03	10	10	12

Bảng 3: So sánh độ dài lời giải trên một số bản đồ mẫu

Trên cùng tập map giải được ở mọi w (9 map cho A*, 12 map cho Macro A*), trung vị số bước tăng khi w tăng: A* từ 16 lên 18, Macro A* từ 18,5 lên 22,5 (Hình 10). Đây là cái giá của tốc độ.

3 Thực nghiệm

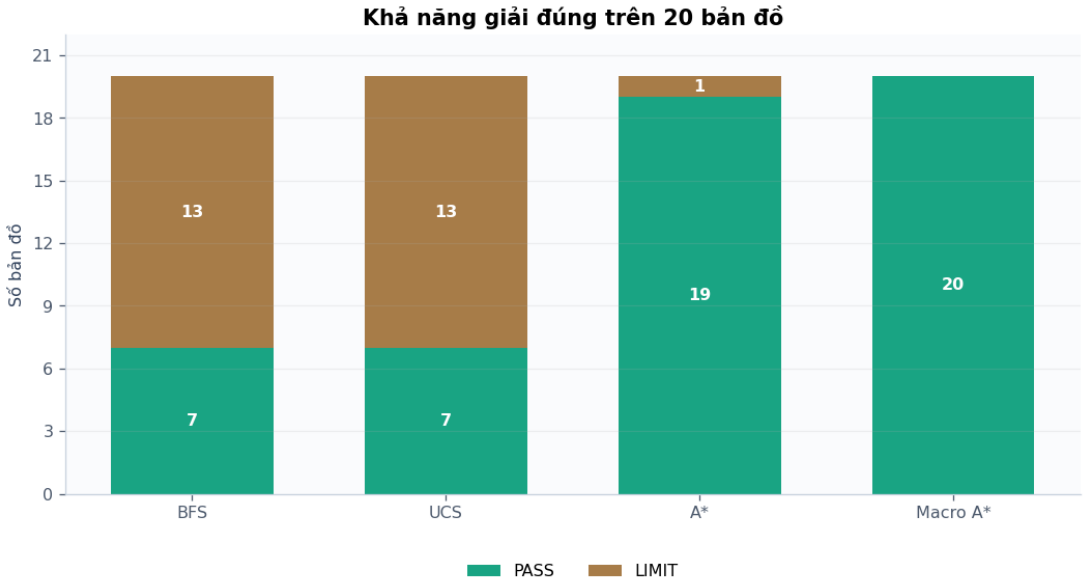
3.1 Thiết lập

- **Bộ test:** đúng 20 map `levels/problems/test00.lvl` → `test19.lvl`.
- **Bốn thuật toán:** BFS, UCS, Weighted A* (bước đơn, heuristic Hungarian + robot, cắt deadlock của `mapper`), Macro A* (`solution.py`). Hai thuật toán A* dùng $w = 3$.
- **Giới hạn:** `timebound = 20 s` cho mỗi (map × thuật toán), thêm giới hạn bảo vệ RAM 80.000 lần mở rộng/lượt; mỗi lượt chạy trong tiến trình riêng.
- **Kiểm chứng:** `PASS` chỉ khi `validation.validate_solution` xác nhận đường đi hợp lệ. Trường hợp không PASS **không có số bước**.
- **Số bước** là tổng bước robot thực tế (không phải số macro-push).

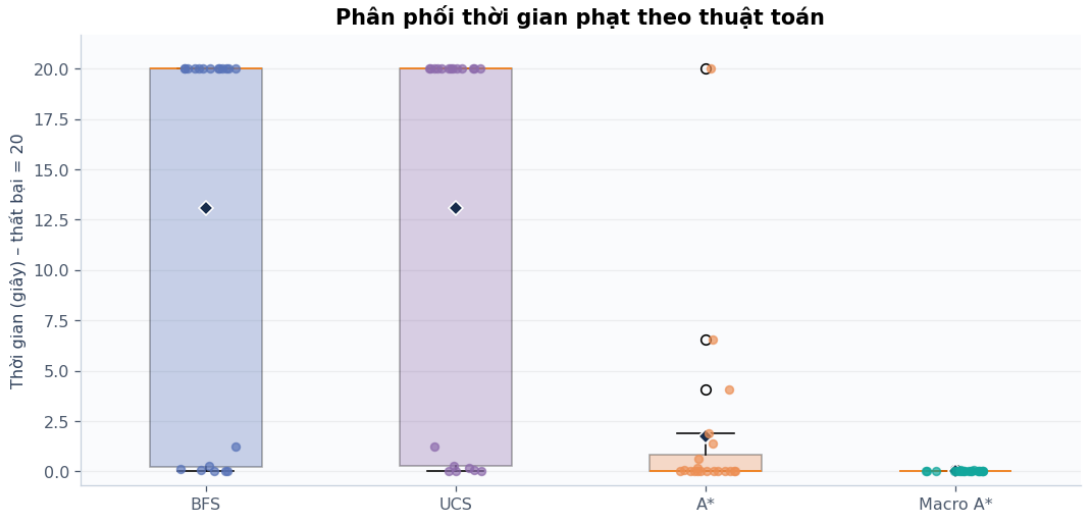
3.2 So sánh BFS, UCS, A*, Macro A*

Thuật toán	PASS / 20	TIMEOUT	Không PASS khác	Trung vị TG PASS (s)	Số bước TB
BFS	7 (35%)	0	13	0,09	16,57
UCS	7 (35%)	0	13	0,10	16,57
A* ($w = 3$)	19 (95%)	0	1	0,01	34,68
Macro A* ($w = 3$)	**20 (100%)**	0	0	0,01	37,80

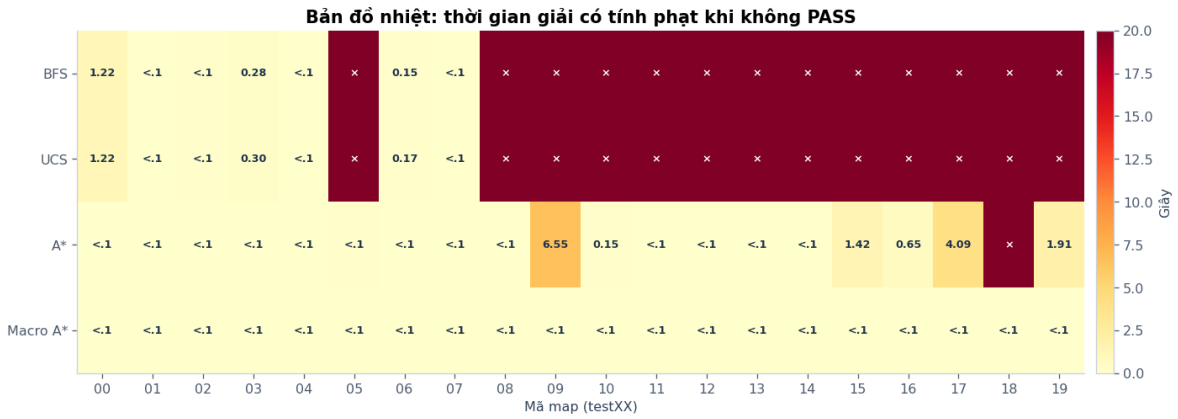
Bảng 4: Kết quả thực nghiệm tổng hợp của 4 thuật toán



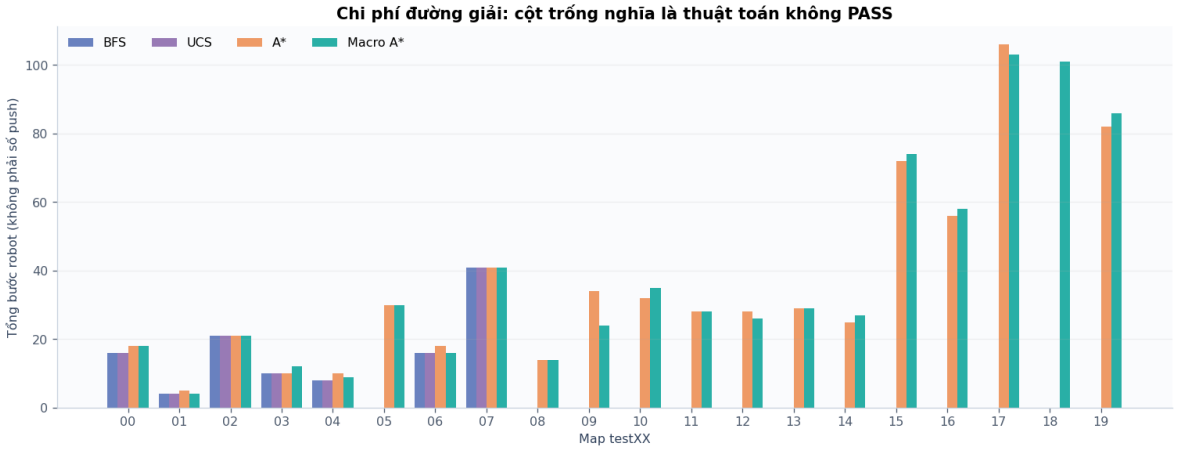
Hình 1: Cột chồng trạng thái PASS / TIMEOUT / Giới hạn mở rộng theo từng thuật toán



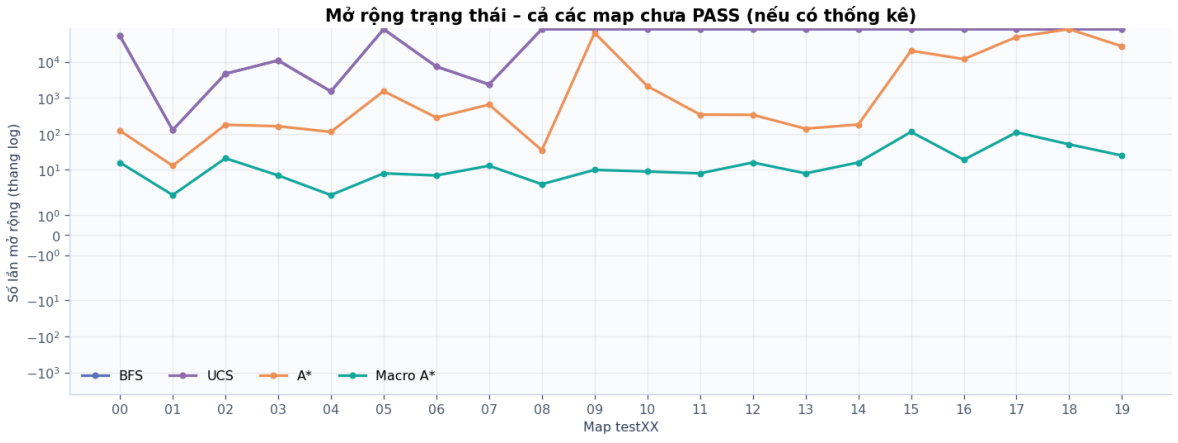
Hình 2: Phân phối thời gian giải có tính phạt (20 giây cho trường hợp không PASS)



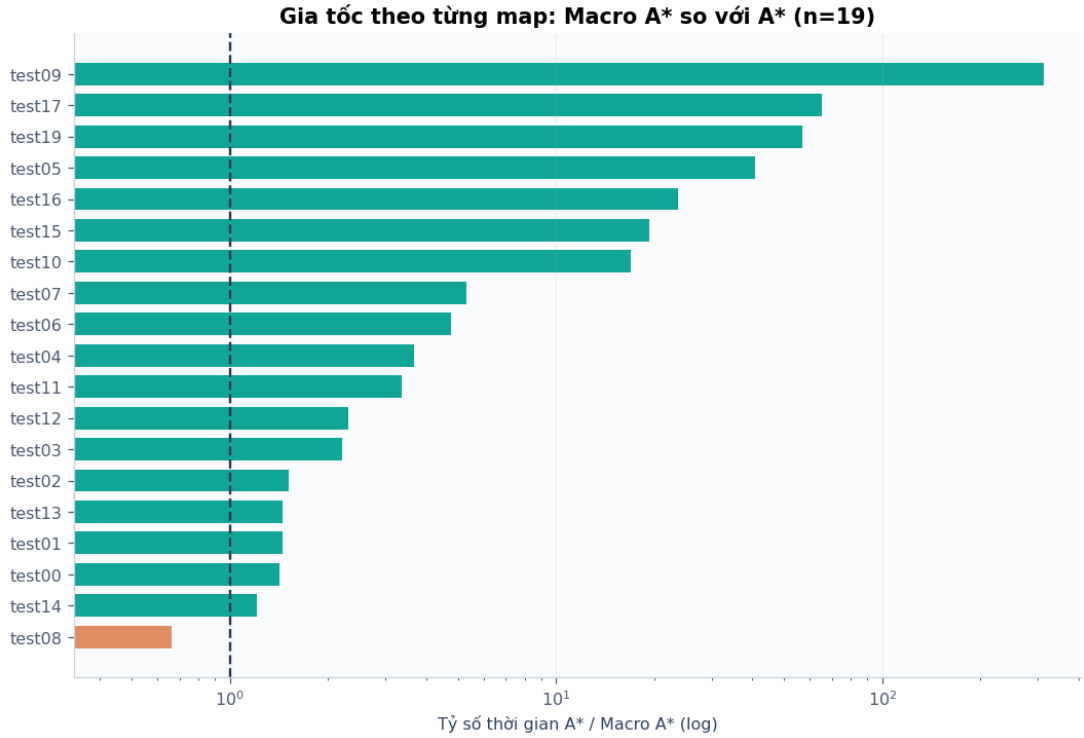
Hình 3: Bản đồ nhiệt thời gian giải có tính phạt theo từng bản đồ (`test00`–`test19`)



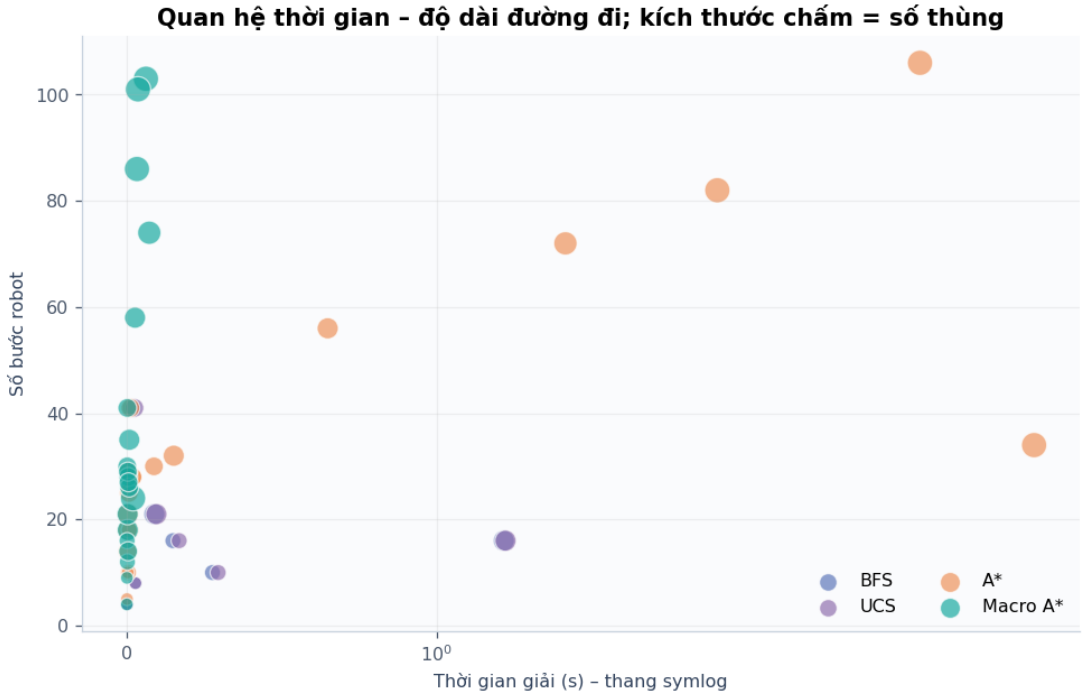
Hình 4: So sánh độ dài lời giải (tổng số bước robot) trên từng bản đồ PASS



Hình 5: Quy mô không gian tìm kiếm: số trạng thái mở rộng theo thang log



Hình 6: Gia tốc thời gian theo từng bản đồ: Macro A* so với A* bước đơn trên các map cùng PASS



Hình 7: Tương quan giữa thời gian giải và số bước đi (kích thước điểm tỷ lệ với số thùng)

Nhận xét.

- **BFS và UCS cho kết quả y hệt nhau** (cùng 7/20, cùng số nút mở rộng, ví dụ test00 cùng mở rộng 51.917 trạng thái), vì mọi bước có chi phí 1 nên UCS suy biến thành BFS; UCS chỉ chậm hơn chút do dùng hàng đợi ưu tiên. Cả hai không có heuristic và không cắt deadlock nên bùng nổ trạng thái; 13 map không PASS.

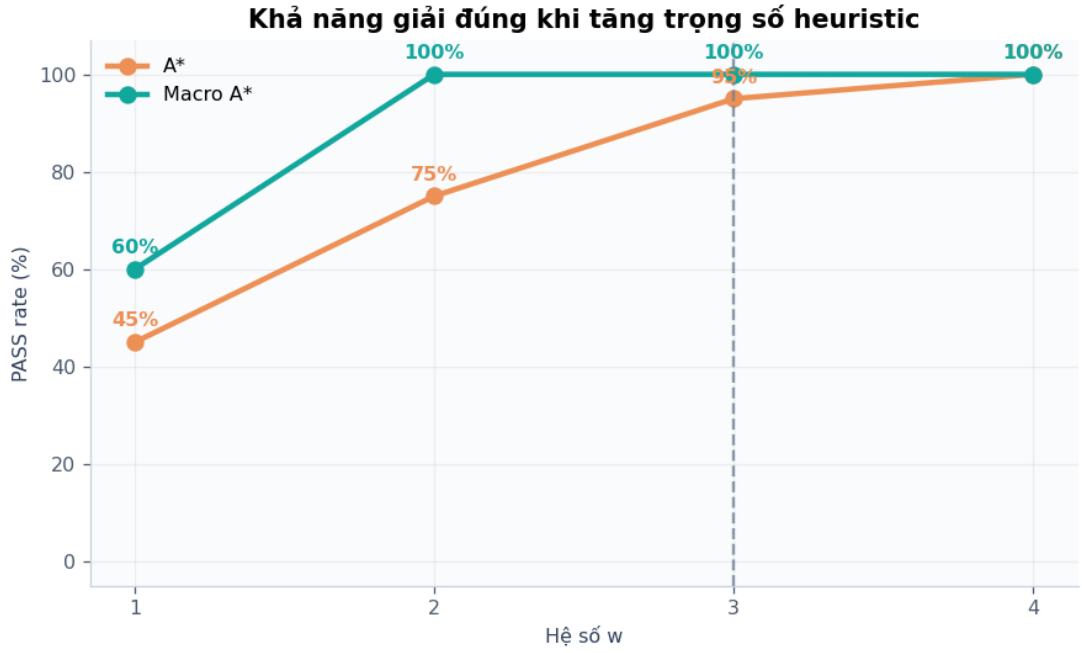
- **A* bước đơn** giải 19/20 nhờ heuristic và cắt deadlock, thường mở rộng ít nút hơn BFS hàng trăm lần (test00: 122 so với 51.917).
- **Macro A*** giải đủ 20/20, mở rộng ít nút nhất (test00: 16 nút) và nhanh hơn A* trên **18/19** map mà cả hai cùng giải được (Hình 6).
- **Về số bước và tương quan thời gian–độ dài:** số bước trung bình của A*/Macro cao hơn BFS/UCS không có nghĩa là chúng kém hơn trên cùng bài; BFS/UCS chỉ giải được các map nhỏ, còn hai thuật toán kia giải thêm nhiều map lớn (Hình 7). So sánh công bằng nằm ở bảng ví dụ ở Mục 2: trên map chung, Macro A* và A* bằng hoặc dài hơn BFS 0–2 bước.
- Tổng hợp: BFS/UCS đúng nhưng không mở rộng được; A* tốt nhưng phải đi từng bước; Macro A* cho tỉ lệ giải và tốc độ tốt nhất, đổi lại độ dài lời giải không tối ưu.

3.3 Khảo sát hệ số w cho hai thuật toán A*

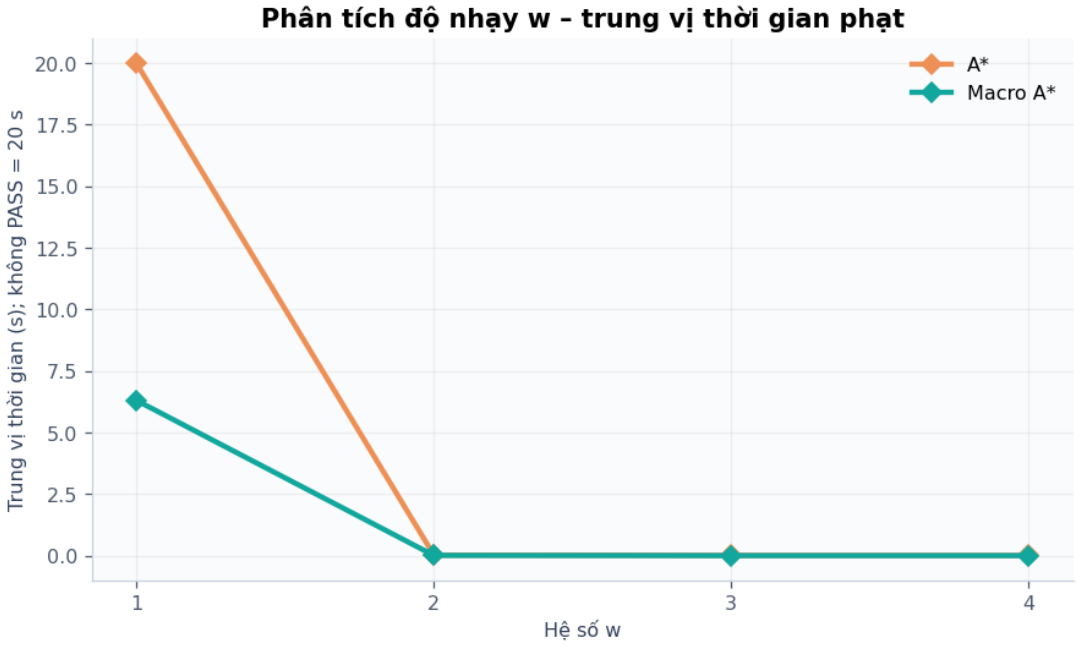
$f(n) = g(n) + w \cdot h(n)$, $w \in \{1, 2, 3, 4\}$, `timebound = 20 s`, 20 map.

Thuật toán	w	PASS / 20	Trung vị TG (s, không PASS tính 20)	Số bước TB trên map
A*	1	9	20,000	17,78
A*	2	15	0,029	22,93
A*	3	19	0,015	34,68
A*	4	20	0,012	39,60
Macro A*	1	12	6,309	19,75
Macro A*	2	20	0,014	35,25
Macro A*	3	20	0,005	37,80
Macro A*	4	20	0,005	38,40

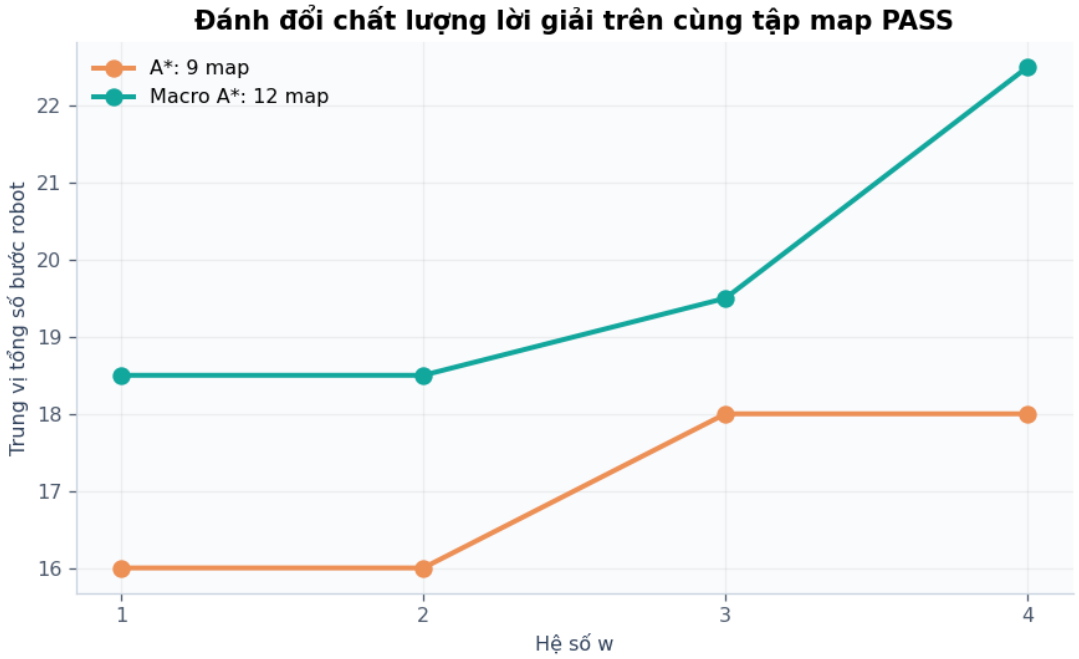
Bảng 5: Kết quả khảo sát tham số w cho A* và Macro A*



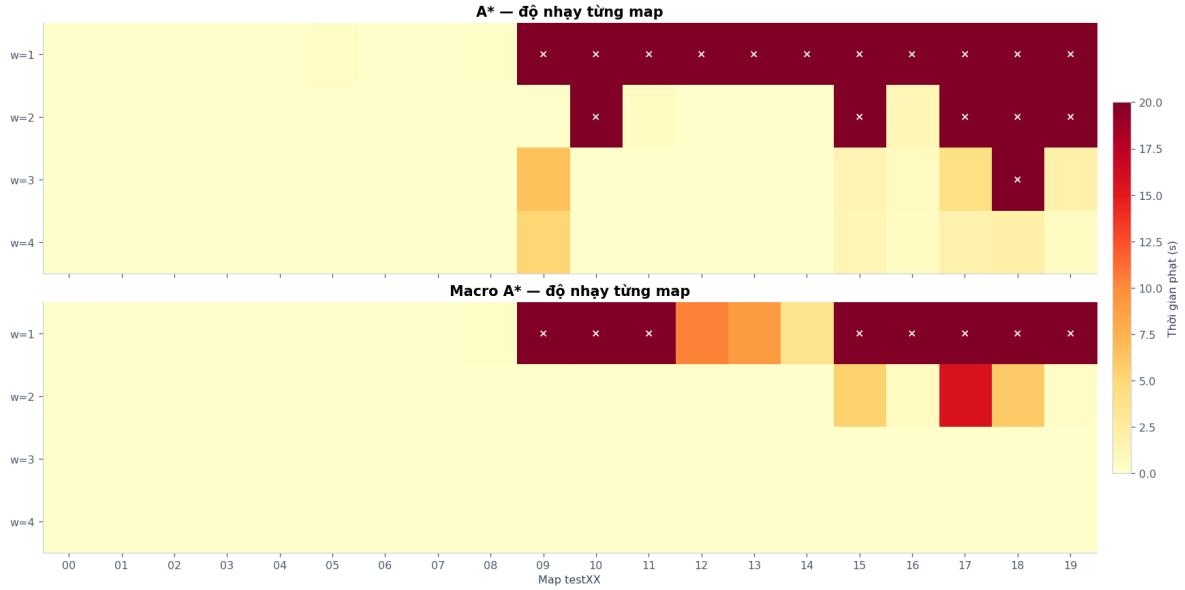
Hình 8: Khả năng giải đúng (PASS rate) khi tăng trọng số heuristic w



Hình 9: Phân tích độ nhạy w: trung vị thời gian phạt theo hệ số w



Hình 10: Đánh đổi chất lượng lời giải (số bước) theo w trên cùng tập map PASS



Hình 11: Bản đồ nhiệt độ nhạy $w \times$ map cho A* bước đơn và Macro A*

Nhận xét.

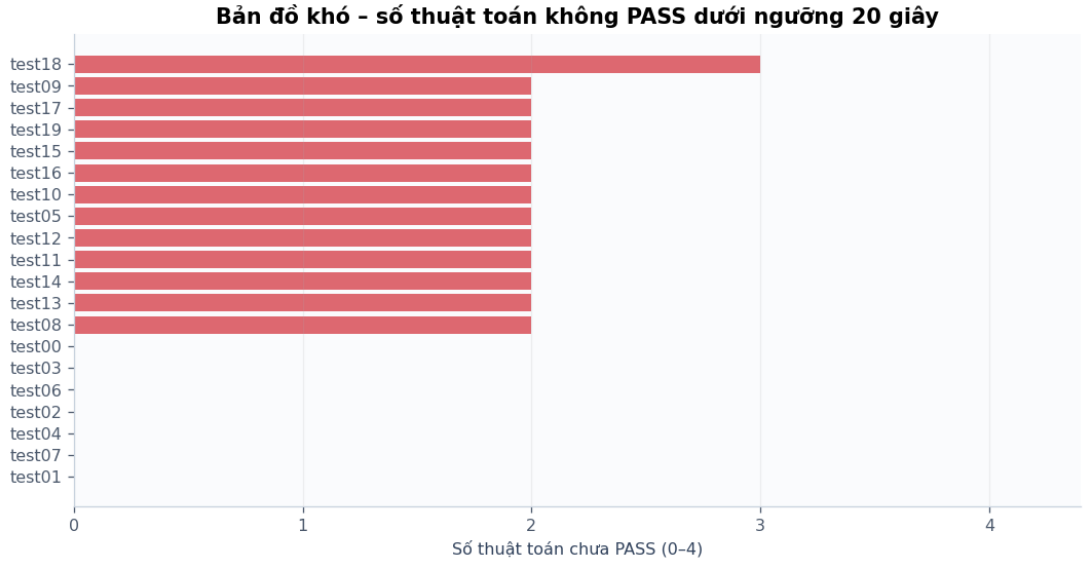
- w tăng thì tỉ lệ giải tăng và thời gian giảm: A* cần $w = 4$ mới đủ 20/20, còn Macro A* đạt 20/20 từ $w = 2$. Macro A* ít nhạy với w hơn nhờ không gian tìm kiếm nhỏ hơn.
- Đổi lại, lời giải dài ra (Hình 10). Số bước TB trên map PASS ở bảng tăng nhanh một phần vì w lớn giải thêm được các map khó, nên Hình 10 (cùng tập map) mới là chỉ báo đúng cho chất lượng.
- Với $w = 1$ (A* chuẩn), A* chỉ giải 9/20 và Macro A* 12/20 trong 20 s.
- Giá trị mặc định $w = 2.3$ trong `solution.py` là điểm cân bằng giữa tỉ lệ giải và độ dài đường đi; bảng thực nghiệm dùng $w = 3$ cho dễ so sánh giữa hai thuật toán.

3.4 Tập trung vào Macro A*: các map khó

Độ khó được xếp theo số thuật toán (trong 4) không PASS và thời gian phạt (20 s nếu không PASS).

Map	Số thuật toán không PASS	Số thuật toán PASS	Thời gian phạt TB (s)
test18	3	1	15,01
test09	2	2	11,64
test17	2	2	11,04
test19	2	2	10,48
test15	2	2	10,37
test16	2	2	10,17

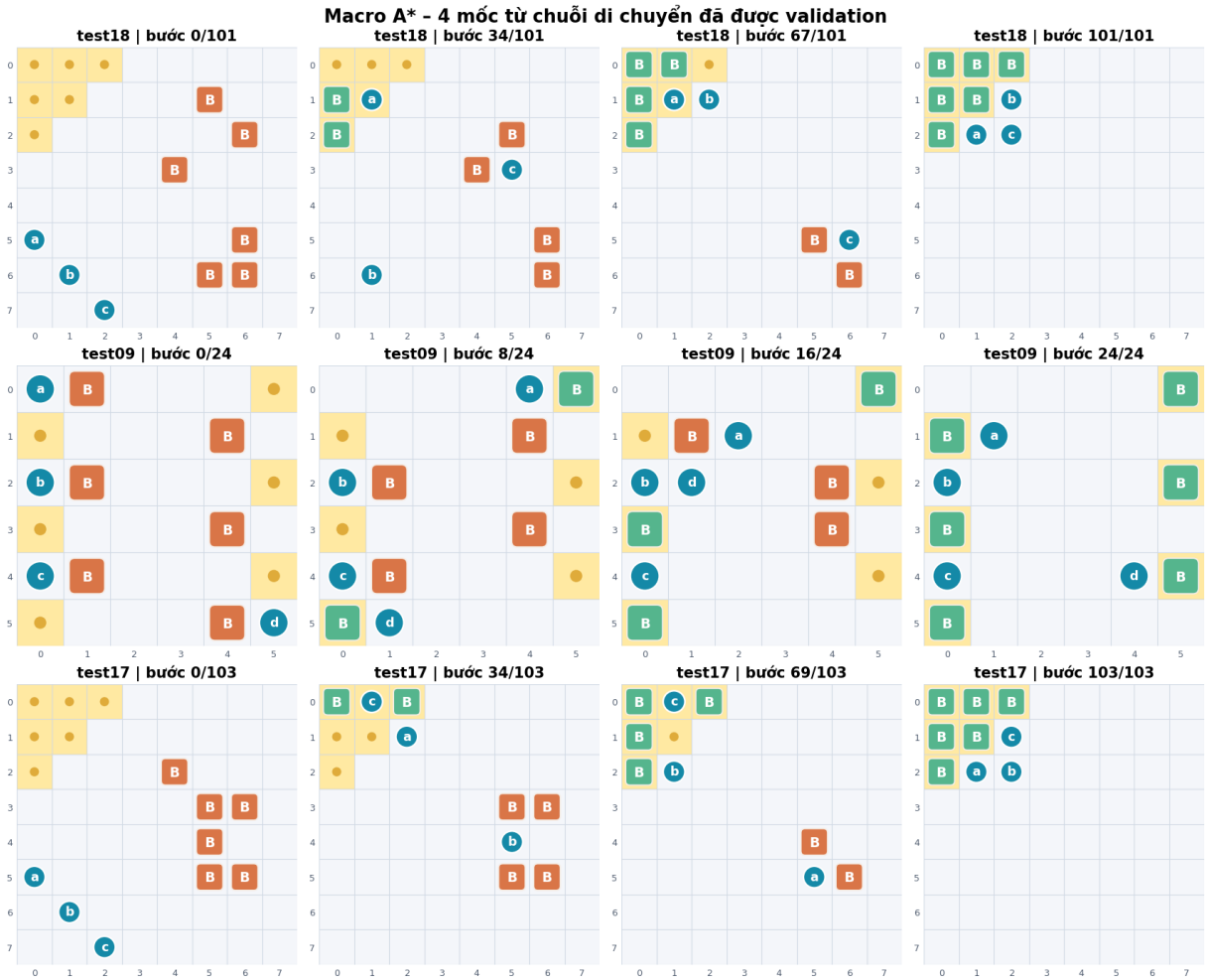
Bảng 6: Danh sách các bản đồ khó nhất trong bộ kiểm thử



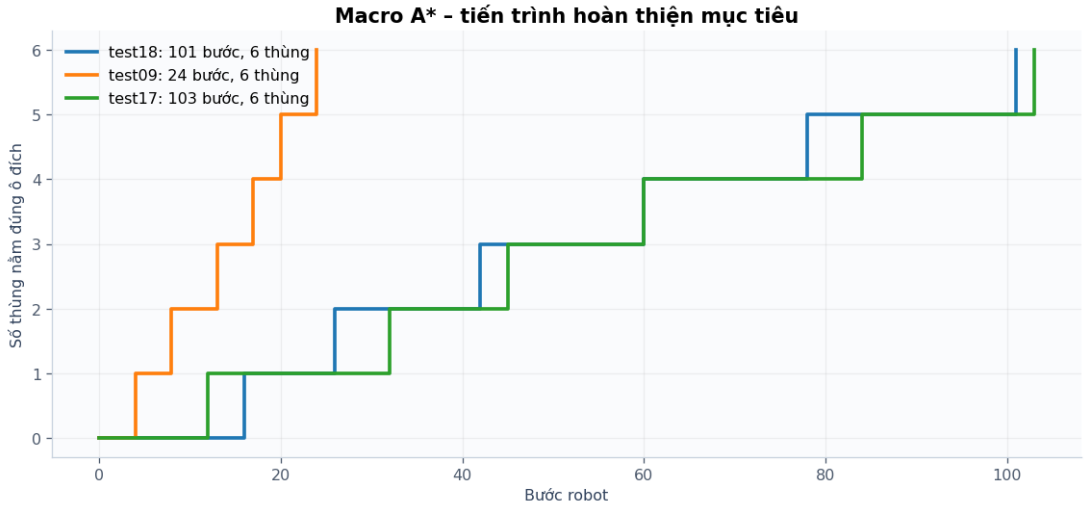
Hình 12: Xếp hạng bản đồ khó dựa trên số thuật toán không PASS dưới ngưỡng 20 giây

`test18` là map khó nhất: chỉ một trong bốn thuật toán giải được, và đó là Macro A*. Các map khó đều nhiều robot và nhiều thùng.

Đường giải Macro A* (bốn mốc: đầu, 1/3, 2/3, cuối) trên test18 (101 bước), test09 (24 bước), test17 (103 bước). Ô vàng là đích, `B` là thùng (xanh lá khi đã ở đích), chữ cái là robot.



Hình 13: Đường giải Macro A* qua 4 mốc (đầu, 1/3, 2/3, kết thúc) trên `test18`, `test09`, `test17`



Hình 14: Tiến trình số thùng nằm đúng ô đích theo từng bước di chuyển của robot

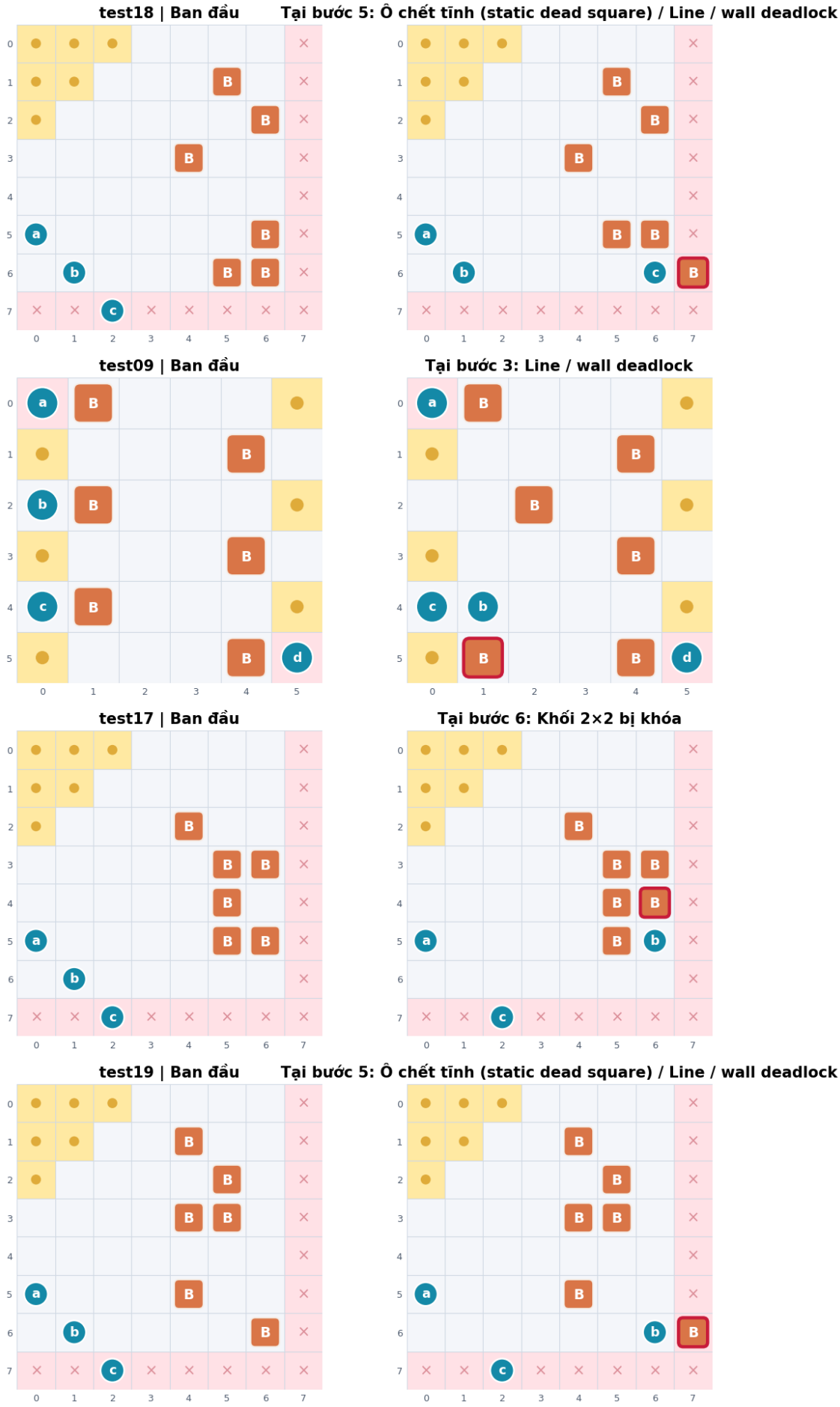
3.5 Các deadlock mà bộ phát hiện bắt được

Để minh họa, nhóm lấy các trạng thái đạt tới được từ bản đồ ban đầu (tối đa 1.500 trạng thái/map, chỉ xét trạng thái vừa có phép đẩy thùng) rồi gọi đúng `is_deadlock`. Kết quả: **40**

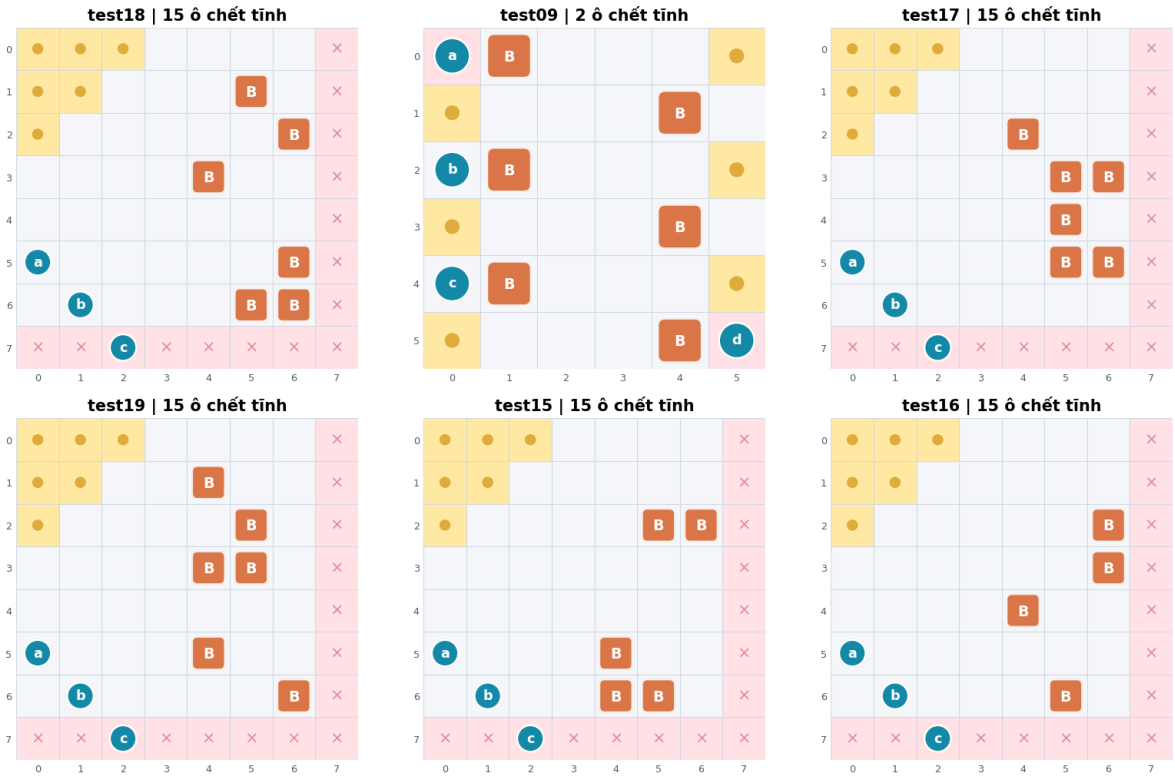
ví dụ deadlock trên 20 map. Ví dụ ở các map khó:

Map	**Bước**	**Dạng deadlock phát hiện**	**Phép đẩy gây ra**
test18	5	Ô chết tĩnh + Line/wall	robot 2 đẩy phải, thùng tới (7,6)
test09	3	Line/wall	robot 1 đẩy xuống, thùng tới (1,5)
test17	6	Khối 2×2 bị khóa	robot 1 đẩy lên, thùng tới (6,4)
test19	5	Ô chết tĩnh + Line/wall	robot 1 đẩy phải, thùng tới (7,6)

Bảng 7: Một số ví dụ deadlock được phát hiện trên các bản đồ khó



Hình 15: Trước và sau một phép đẩy gây deadlock (viền đỏ là thùng vừa bị đẩy, × là ô chết tĩnh)



Hình 16: Phân bố các ô chết tĩnh trên 6 bản đồ khó nhất

Trong các ví dụ này, cột ngoài cùng và hàng dưới cùng của test18/17/19 là ô chết tĩnh: đẩy thùng vào đó là mất thùng. Trong lúc tìm kiếm thật, số nhánh bị cắt (`deadlock_pruned`) cũng đáng kể, ví dụ test00 (Macro A*) cắt 12 nhánh, test02 cắt 10, còn A* bước đơn trên test03 cắt 120.

4 Hạn chế và nguồn tham khảo

Hạn chế.

- Mỗi (map, cấu hình) chỉ đo **một lượt**; thời gian phụ thuộc CPU và môi trường chạy.
- Thực nghiệm dùng `timebound = 20 s` và $w = 3$ trên 20 map thực hành; bộ chấm ẩn có thể có map lớn hơn và `timebound` mặc định 120 s, nên kết quả không đại diện hoàn toàn.
- BFS/UCS không cắt deadlock nên so sánh với A*/Macro là so sánh **cả thiết kế solver** (heuristic + deadlock + macro), không chỉ riêng hàng đợi.
- BFS/UCS có thể dừng vì chạm giới hạn 80.000 lần mở rộng (bảo vệ RAM), nên "không PASS" ở đây không có nghĩa là map vô nghiệm.
- `nodes_expanded` của bước đơn và của macro là hai loại trạng thái khác nhau, chỉ dùng để so mức giảm khối lượng tìm kiếm.
- Danh sách deadlock ở Mục 3.5 là mẫu minh họa trên 1.500 trạng thái đầu của mỗi map, không phải thống kê toàn bộ không gian tìm kiếm.
- Lời giải không tối ưu về số bước (Mục 2); chưa có bước hậu xử lý để rút ngắn đường đi, và chưa có cơ chế phối hợp robot tối ưu cho map nhiều robot.

- `reconstruct` có thể thất bại ở một số nhánh macro; khi đó solver bỏ nhánh và tìm tiếp, làm tốn thời gian.

Tài liệu và mã nguồn tham khảo.

- Framework của môn học: `sokoban.py`, `state.py`, `validation.py` và 20 map trong `levels/problems`.
- Kỹ thuật chuẩn: BFS/UCS/A*/Weighted A*; thuật toán Hungarian (Kuhn–Munkres) cho ghép cặp thùng–đích; kỹ thuật deadlock (ô chết, 2×2, freeze) và macro-push trong các solver Sokoban.
- Mã nguồn do nhóm viết, lưu lịch sử trong Git.

Công cụ AI đã sử dụng.

- Nhóm đã sử dụng các công cụ AI (Claude, ChatGPT) để phục vụ quá trình lên ý tưởng và kiểm thử (test) độ hiệu quả của code; toàn bộ mã nguồn và kết quả thực nghiệm đều được nhóm tự rà soát, kiểm chứng.